Perfect. Let’s extend your Planetary Project into a fully integrated Planetary Pulse Engine. This is where aura_core, Lucy, the dashboard, and live simulations all run off the same real-time state—like the “heartbeat” of civilization progress.

---

1. Core Idea

aura_core = immutable math engine (KTE, TT1, CEI, coherence, transition differential)

Digital Twins = two parallel Earths: historical baseline & AI-accelerated scenario

Lucy = interface + reasoning + explainability

Dashboard / Sonification = live visualization of progress, coherence, and risk metrics

Every agent / dashboard / Lucy references one source of truth: the aura_core simulation state.

---

2. Folder Layout (Planetary Pulse Engine)

```
planetary/
├── aura_core/                # Locked math + twin Earths
│   ├── equations/
│   │   ├── KTE_equation.py
│   │   ├── CEI_equation.py
│   │   ├── SKI_equation.py
│   │   ├── transition_rate.py
│   │   ├── time_to_type1.py
│   │   ├── co_evo_ledger.py
│   │   ├── transition_diff.py
│   │   └── coherence_equation.py
│   ├── twins/
│   │   ├── Earth1_historical/
│   │   └── Earth2_accelerated/
│   ├── constants.py
│   └── __init__.py
├── causality_engine/
│   └── attribution_model.py
├── collective_intelligence/
```

```
│   └── sentiment_analysis.py
├── macro_simulation/
│   └── dynamics.py
├── governance/
│   ├── intervention_registry.py
│   ├── intervention_rules.py
│   ├── audit_ledger.py
│   └── middleware.py
├── data_validation/
│   └── integrity_checks.py
├── dashboard/
│   ├── app.py
│   ├── plots.py
│   └── sonification.py
├── lucy_interface/
│   ├── lucy_core.py
│   ├── planner.py
│   ├── interface_manager.py
│   ├── eagle_eyes.py
│   ├── task_executor.py
│   ├── memory.py
│   └── voice_visual.py
├── data_inputs/
│   ├── streams.py
│   └── api_clients.py
├── autonomy_orchestrator/
│   └── dispatcher.py
├── risk_management/
│   └── preflight_checks.py
└── training_tools/
    └── replay_ui.py
```

---

3. How Equations Are Calculated Per Agent

Energy Agent → modifies $T_i$ only

AI Agent → modifies $A_i$ only

Policy Agent → modifies $P_i$ only

Collective Intelligence Agent → adjusts C (coherence multiplier)

Agents never touch K directly.

Flow:

Agent Proposal → validated by governance → aura_core computes KTE → updates simulation state → feedback loop to agents

---

4. Lucy Integration with aura_core

Lucy is the orchestrator / explanation layer:

1. Planner → receives task / question

2. Interface Manager → converts into calls to aura_core or macro_simulation

3. Eagle Eyes → monitors agent execution and checks for forbidden interventions

4. Memory / Reward → keeps historical records of simulations for reasoning

5. Voice / Visual → outputs dashboard-ready info

Lucy doesn't modify equations directly — she queries the state.

Logic flow example:

User → Lucy → Orchestrator → aura_core.simulate_step()
↘ Causality Engine → explanation
↘ Dashboard → visualization
Lucy → interprets & communicates results → user

---

5. Real-Time Planetary Pulse

Simulation State lives in aura_core

Agents propose changes → aura_core validates and applies equations

Lucy queries state → explains projections

Dashboard updates → K(t), TT1, CEI, coherence, risk metrics, sonification

Coherence Meter + Monte Carlo bands → show uncertainty and global coordination index

Key Concept: everyone references the same K(t) vector in memory. No agent can bypass rules.

---

6. Forbidden Intervention Enforcement

Middleware / Audit Ledger = prevents agents from making unsafe proposals

Lucy = can explain why a proposal is blocked, never executes blocked actions

FORB_REGISTRY = maps 102 forbidden codes → blocks unsafe actions in real time

Example:

```
check = validate_action("population_control_coercive")
if check["status"] == "BLOCKED":
    Lucy.explain_blocked(check)
```

---

7. Project Blueprint Summary

| Layer | Role |
|---|---|
| aura_core | Math engine, simulations, digital twins |
| causality_engine | Attribution / explainable reasoning |
| collective_intelligence | Global sentiment / coordination input |
| macro_simulation | Long-horizon scenario projections |

| | |
|---|---|
| governance | Middleware, forbidden interventions, audit logs |
| data_validation | Ensure all streams & inputs are reliable |
| dashboard | Visualization & sonification (Planetary Pulse) |
| lucy_interface | Planner, reasoning, explanation, feedback |
| data_inputs | Real-time API / streaming data |
| autonomy_orchestrator | Agent dispatch, parallel execution |
| risk_management | Preflight checks, failsafe policies |
| training_tools | Replay, educational front-end |

---

✅ Summary

Single source of truth = aura_core simulation state

Agents propose & adjust only parameters → K computed centrally

Lucy interprets & explains → never breaks governance

Dashboard visualizes & sonifies → live planetary pulse

Forbidden interventions enforced in middleware → full safety

---

If you want, I can make a diagram showing the full flow: aura_core, agents, Lucy, dashboard, and pulse engine, with all interactions clearly labeled.

Do you want me to make that diagram next?